Triangle loop algoritmus

https://gemini.google.com/app/a13349fd82a7e09d

*User prompt: import java.util.HashMap; import java.util.ArrayList; // Eredeti és aktuális adatok ArrayList<PVector> baseVertices; ArrayList<ArrayList<Integer>> baseFaces; ArrayList<PVector> currentVertices; ArrayList<ArrayList<Integer>> currentFaces; int subLevel = 0; // GPU-gyorsított alakzat a sima forgatáshoz PShape meshShape; // Forgatási szögek, zoom és hálózat kapcsoló float rotX = 0; float rotY = 0; float zoom = 1.0f; boolean showWireframe = true; void settings() { size(800, 600, P3D); } void setup() { surface.setLocation(100, 100); frameRate(30); noLoop(); // Megállítja az alap animációs ciklust loadOBJ("rectangle.obj"); rebuildMesh(); } void loadOBJ(String filename) { baseVertices = new ArrayList<PVector>(); baseFaces = new ArrayList<ArrayList<Integer>>(); String[] lines = loadStrings(filename); if (lines == null) { println("Hiba: Nem található a fájl: " + filename); return; } for (String line : lines) { line = line.trim(); if (line.startsWith("v ")) { String[] parts = line.split("\\s+"); float x = float(parts[1]); float y = float(parts[2]); float z = float(parts[3]); baseVertices.add(new PVector(x, y, z)); } else if (line.startsWith("f ")) { String[] parts = line.split("\\s+"); ArrayList<Integer> face = new ArrayList<Integer>(); for (int i = 1; i < parts.length; i++) { String[] indices = parts[i].split("/"); int vIndex = Integer.parseInt(indices[0]) - 1; face.add(vIndex); } baseFaces.add(face); } } centerAndScaleModel(); } void centerAndScaleModel() { if (baseVertices.isEmpty()) return; PVector center = new PVector(0, 0, 0); for (PVector v : baseVertices) { center.add(v); } center.div(baseVertices.size()); for (PVector v : baseVertices) { v.sub(center); } float maxDist = 0; for (PVector v : baseVertices) { float d = v.mag(); if (d > maxDist) maxDist = d; } if (maxDist > 0) { float scaleFactor = 150.0f / maxDist; for (PVector v : baseVertices) { v.mult(scaleFactor); } } } void keyPressed() { if (key == '1') { subLevel = constrain(subLevel + 1, 0, 4); rebuildMesh(); redraw(); } else if (key == '2') { subLevel = constrain(subLevel - 1, 0, 4); rebuildMesh(); redraw(); } else if (key == '3') { showWireframe = !showWireframe; updateMeshShape(); // Frissítjük a háló megjelenítést a PShape-en belül is redraw(); } else if (key == CODED) { if (keyCode == UP) { zoom += 0.1f; redraw(); } else if (keyCode == DOWN) { zoom = max(0.1f, zoom - 0.1f); redraw(); } } } void mouseDragged() { float dx = mouseX - pmouseX; float dy = mouseY - pmouseY; rotY += dx * 0.01f; rotX -= dy * 0.01f; redraw(); // Mostantól villámgyors lesz a PShape miatt! } void rebuildMesh() { currentVertices = new ArrayList<PVector>(); for (PVector v : baseVertices) currentVertices.add(v.copy()); currentFaces = new ArrayList<ArrayList<Integer>>(); for (ArrayList<Integer> f : baseFaces) { ArrayList<Integer> nf = new ArrayList<Integer>(); for (int idx : f) nf.add(idx); currentFaces.add(nf); } for (int i = 0; i < subLevel; i++) { catmullClarkStep(); } // A hálózat módosülése után generáljuk újra a GPU alakzatot updateMeshShape(); } // Catmull-Clark lépés (változatlan marad) void catmullClarkStep() { int numVerts = currentVertices.size(); int numFaces = currentFaces.size(); ArrayList<PVector> facePoints = new ArrayList<PVector>(); for (int i = 0; i < numFaces; i++) { ArrayList<Integer> f = currentFaces.get(i); PVector fp = new PVector(0, 0, 0); for (int idx : f) { fp.add(currentVertices.get(idx)); } fp.div(f.size()); facePoints.add(fp); } HashMap<String, ArrayList<Integer>> edgeFaces = new HashMap<String, ArrayList<Integer>>(); for (int fIdx = 0; fIdx < numFaces; fIdx++) { ArrayList<Integer> f = currentFaces.get(fIdx); int size = f.size(); for (int i = 0; i < size; i++) { int u = f.get(i); int v = f.get((i + 1) % size); String key = Math.min(u, v) + "_" + Math.max(u, v); if (!edgeFaces.containsKey(key)) { edgeFaces.put(key, new ArrayList<Integer>()); } edgeFaces.get(key).add(fIdx); } } HashMap<String, PVector> edgePoints = new HashMap<String, PVector>(); for (String key : edgeFaces.keySet()) { String[] parts = key.split("_"); int u = Integer.parseInt(parts[0]); int v = Integer.parseInt(parts[1]); PVector p1 = currentVertices.get(u);*

PVector p2 = currentVertices.get(v); PVector ep = new PVector(0, 0, 0); ep.add(p1); ep.add(p2); ArrayList<Integer> adjFaces = edgeFaces.get(key); for (int fIdx : adjFaces) { ep.add(facePoints.get(fIdx)); } ep.div(2.0f + adjFaces.size()); edgePoints.put(key, ep); } ArrayList<ArrayList<Integer>> vertexFaces = new ArrayList<ArrayList<Integer>>(); ArrayList<ArrayList<String>> vertexEdges = new ArrayList<ArrayList<String>>(); for (int i = 0; i < numVerts; i++) { vertexFaces.add(new ArrayList<Integer>()); vertexEdges.add(new ArrayList<String>()); } for (int fIdx = 0; fIdx < numFaces; fIdx++) { ArrayList<Integer> f = currentFaces.get(fIdx); for (int idx : f) { if (!vertexFaces.get(idx).contains(fIdx)) { vertexFaces.get(idx).add(fIdx); } } } for (String key : edgeFaces.keySet()) { String[] parts = key.split("_"); int u = Integer.parseInt(parts[0]); int v = Integer.parseInt(parts[1]); if (!vertexEdges.get(u).contains(key)) vertexEdges.get(u).add(key); if (!vertexEdges.get(v).contains(key)) vertexEdges.get(v).add(key); } ArrayList<PVector> newVertexPositions = new ArrayList<PVector>(); for (int i = 0; i < numVerts; i++) { PVector oldV = currentVertices.get(i); ArrayList<Integer> vFaces = vertexFaces.get(i); ArrayList<String> vEdges = vertexEdges.get(i); int n = vEdges.size(); if (n == 0) { newVertexPositions.add(oldV.copy()); continue; } PVector Q = new PVector(0, 0, 0); for (int fIdx : vFaces) { Q.add(facePoints.get(fIdx)); } Q.div(vFaces.size()); PVector R = new PVector(0, 0, 0); for (String eKey : vEdges) { String[] parts = eKey.split("_"); int u = Integer.parseInt(parts[0]); int v = Integer.parseInt(parts[1]); PVector other = (u == i) ? currentVertices.get(v) : currentVertices.get(u); PVector mid = PVector.add(oldV, other).mult(0.5f); R.add(mid); } R.div(n); PVector newV = new PVector(0, 0, 0); newV.add(Q); newV.add(PVector.mult(R, 2.0f)); newV.add(PVector.mult(oldV, n - 3)); newV.div(n); newVertexPositions.add(newV); } ArrayList<PVector> nextVertices = new ArrayList<PVector>(); for (PVector v : newVertexPositions) { nextVertices.add(v); } int facePointStartIndex = nextVertices.size(); for (PVector fp : facePoints) { nextVertices.add(fp); } HashMap<String, Integer> edgePointIndices = new HashMap<String, Integer>(); for (String key : edgePoints.keySet()) { edgePointIndices.put(key, nextVertices.size()); nextVertices.add(edgePoints.get(key)); } ArrayList<ArrayList<Integer>> nextFaces = new ArrayList<ArrayList<Integer>>(); for (int fIdx = 0; fIdx < numFaces; fIdx++) { ArrayList<Integer> f = currentFaces.get(fIdx); int fpIdx = facePointStartIndex + fIdx; int size = f.size(); for (int i = 0; i < size; i++) { int currV = f.get(i); int nextV = f.get((i + 1) % size); int prevV = f.get((i - 1 + size) % size); String edgeCurr = Math.min(currV, nextV) + "_" + Math.max(currV, nextV); String edgePrev = Math.min(currV, prevV) + "_" + Math.max(currV, prevV); int epCurr = edgePointIndices.get(edgeCurr); int epPrev = edgePointIndices.get(edgePrev); ArrayList<Integer> newQuad = new ArrayList<Integer>(); newQuad.add(currV); newQuad.add(epCurr); newQuad.add(fpIdx); newQuad.add(epPrev); nextFaces.add(newQuad); } } currentVertices = nextVertices; currentFaces = nextFaces; } // --- ÚJ: A PShape felépítése (Háromszögelve a GPU számára) --- void updateMeshShape() { meshShape = createShape(); meshShape.beginShape(TRIANGLES); if (showWireframe) { meshShape.stroke(100, 200, 255); } else { meshShape.noStroke(); } meshShape.fill(40, 90, 140, 220); for (ArrayList<Integer> f : currentFaces) { int size = f.size(); if (size == 4) { // Quad feldarabolása 2 darab háromszögre int v0 = f.get(0); int v1 = f.get(1); int v2 = f.get(2); int v3 = f.get(3); addVert(v0); addVert(v1); addVert(v2); addVert(v0); addVert(v2); addVert(v3); } else if (size == 3) { addVert(f.get(0)); addVert(f.get(1)); addVert(f.get(2)); } else { // Általános poligon fan trianguláció int v0 = f.get(0); for (int i = 1; i < size - 1; i++) { addVert(v0); addVert(f.get(i)); addVert(f.get(i + 1)); } } } meshShape.endShape(); } void addVert(int idx) { PVector v = currentVertices.get(idx); meshShape.vertex(v.x, v.y, v.z); } // Kirajzolásért felelős függvény void draw() { background(30); lights(); pushMatrix(); translate(width / 2, height / 2, 0); scale(zoom); rotateX(rotX); rotateY(rotY); // Itt történik a varázslat: csak egyetlen GPU hívás! if (meshShape != null) { shape(meshShape); } popMatrix(); // --- 2D HUD --- hint(DISABLE_DEPTH_TEST); fill(255); textSize(16); text("Subdivision szint: " + subLevel, 20, 30); text("1-es / 2-es gomb: Szint növelés / csökkentés", 20, 55); text("3-as gomb: Háló / Sima felület váltás", 20, 80); text("Fel/Le nyíl: Közelítés / Távolodás", 20, 105); text("Forgatás: kattints és húzd az egeret", 20, 130); hint(ENABLE_DEPTH_TEST); } **módosítsd a kódot, hogy a triangle_loop algoritmust használja**

Response: Átírtam a kódot úgy, hogy a **Catmull-Clark** algoritmus helyett a háromszög alapú hálókra

optimalizált **Loop subdivision** (Loop-féle élfelező/interpolációs) algoritmust használja.

Mivel a Loop algoritmus szigorúan háromszöges hálókat (triangulated mesh) igényel, a loadOBJ függvény mostantól automatikusan háromszögekre bontja (triangulálja) a betöltött fájl sokszögeit (quadokat vagy poligonokat), így bármilyen .obj fájllal zökkenőmentesen működik.

A módosított Processing kód:

```
import java.util.HashMap;
import java.util.ArrayList;
import java.util.HashSet;

// Eredeti és aktuális adatok
ArrayList<PVector> baseVertices;
ArrayList<ArrayList<Integer>> baseFaces;
ArrayList<PVector> currentVertices;
ArrayList<ArrayList<Integer>> currentFaces;
int subLevel = 0;

// GPU-gyorsított alakzat a sima forgatáshoz
PShape meshShape;

// Forgatási szögek, zoom és hálózat kapcsoló
float rotX = 0;
float rotY = 0;
float zoom = 1.0f;
boolean showWireframe = true;

void settings() {
  size(800, 600, P3D);
}

void setup() {
  surface.setLocation(100, 100);
  frameRate(30);
  noLoop(); // Megállítja az alap animációs ciklust

  loadOBJ("rectangle.obj");
  rebuildMesh();
}

void loadOBJ(String filename) {
  baseVertices = new ArrayList<PVector>();
  baseFaces = new ArrayList<ArrayList<Integer>>();

  String[] lines = loadStrings(filename);
  if (lines == null) {
    println("Hiba: Nem található a fájl: " + filename);
    return;
  }
```

```
  for (String line : lines) {
    line = line.trim();
    if (line.startsWith("v ")) {
      String[] parts = line.split("\\s+");
      float x = float(parts[1]);
      float y = float(parts[2]);
      float z = float(parts[3]);
      baseVertices.add(new PVector(x, y, z));
    } else if (line.startsWith("f ")) {
      String[] parts = line.split("\\s+");
      ArrayList<Integer> face = new ArrayList<Integer>();
      for (int i = 1; i < parts.length; i++) {
        String[] indices = parts[i].split("/");
        int vIndex = Integer.parseInt(indices[0]) - 1;
        face.add(vIndex);
      }

      // Loop subdivision miatt a poligonokat automatikusan háromszögekre
bontjuk (Triangulation)
      if (face.size() == 3) {
        baseFaces.add(face);
      } else if (face.size() == 4) {
        // Quad felbontása 2 háromszögre
        ArrayList<Integer> t1 = new ArrayList<Integer>();
        t1.add(face.get(0)); t1.add(face.get(1)); t1.add(face.get(2));
        baseFaces.add(t1);
        ArrayList<Integer> t2 = new ArrayList<Integer>();
        t2.add(face.get(0)); t2.add(face.get(2)); t2.add(face.get(3));
        baseFaces.add(t2);
      } else if (face.size() > 4) {
        int v0 = face.get(0);
        for (int i = 1; i < face.size() - 1; i++) {
          ArrayList<Integer> t = new ArrayList<Integer>();
          t.add(v0);
          t.add(face.get(i));
          t.add(face.get(i + 1));
          baseFaces.add(t);
        }
      }
    }
  }

  centerAndScaleModel();
}

void centerAndScaleModel() {
  if (baseVertices.isEmpty()) return;

  PVector center = new PVector(0, 0, 0);
  for (PVector v : baseVertices) {
    center.add(v);
  }
```

```
  center.div(baseVertices.size());

  for (PVector v : baseVertices) {
    v.sub(center);
  }

  float maxDist = 0;
  for (PVector v : baseVertices) {
    float d = v.mag();
    if (d > maxDist) maxDist = d;
  }

  if (maxDist > 0) {
    float scaleFactor = 150.0f / maxDist;
    for (PVector v : baseVertices) {
      v.mult(scaleFactor);
    }
  }
}

void keyPressed() {
  if (key == '1') {
    subLevel = constrain(subLevel + 1, 0, 4);
    rebuildMesh();
    redraw();
  } else if (key == '2') {
    subLevel = constrain(subLevel - 1, 0, 4);
    rebuildMesh();
    redraw();
  } else if (key == '3') {
    showWireframe = !showWireframe;
    updateMeshShape();
    redraw();
  } else if (key == CODED) {
    if (keyCode == UP) {
      zoom += 0.1f;
      redraw();
    } else if (keyCode == DOWN) {
      zoom = max(0.1f, zoom - 0.1f);
      redraw();
    }
  }
}

void mouseDragged() {
  float dx = mouseX - pmouseX;
  float dy = mouseY - pmouseY;
  rotY += dx * 0.01f;
  rotX -= dy * 0.01f;

  redraw();
}
```

```
void rebuildMesh() {
  currentVertices = new ArrayList<PVector>();
  for (PVector v : baseVertices) currentVertices.add(v.copy());

  currentFaces = new ArrayList<ArrayList<Integer>>();
  for (ArrayList<Integer> f : baseFaces) {
    ArrayList<Integer> nf = new ArrayList<Integer>();
    for (int idx : f) nf.add(idx);
    currentFaces.add(nf);
  }

  for (int i = 0; i < subLevel; i++) {
    loopSubdivisionStep(); // Itt váltottunk át Loop algoritmusra
  }

  updateMeshShape();
}

// --- ÚJ: Loop Subdivision Algoritmus ---
void loopSubdivisionStep() {
  int numVerts = currentVertices.size();
  int numFaces = currentFaces.size();

  // 1. Élek gyűjtése és szomszédos lapok / csúcsok feltérképezése
  HashMap<String, ArrayList<Integer>> edgeFaces = new HashMap<String,
ArrayList<Integer>>();
  ArrayList<HashSet<Integer>> vertexNeighbors = new
ArrayList<HashSet<Integer>>();
  for (int i = 0; i < numVerts; i++) {
    vertexNeighbors.add(new HashSet<Integer>());
  }

  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    int size = f.size();
    for (int i = 0; i < size; i++) {
      int u = f.get(i);
      int v = f.get((i + 1) % size);

      vertexNeighbors.get(u).add(v);
      vertexNeighbors.get(v).add(u);

      String key = Math.min(u, v) + "_" + Math.max(u, v);
      if (!edgeFaces.containsKey(key)) {
        edgeFaces.put(key, new ArrayList<Integer>());
      }
      edgeFaces.get(key).add(fIdx);
    }
  }

  // 2. Új él-pontok (Odd vertices) számítása a Loop maszk alapján
```

```
  HashMap<String, PVector> edgePoints = new HashMap<String, PVector>();
  for (String key : edgeFaces.keySet()) {
    String[] parts = key.split("_");
    int u = Integer.parseInt(parts[0]);
    int v = Integer.parseInt(parts[1]);
    PVector pA = currentVertices.get(u);
    PVector pB = currentVertices.get(v);

    ArrayList<Integer> adjFaces = edgeFaces.get(key);
    PVector ep = new PVector(0, 0, 0);

    if (adjFaces.size() == 2) {
      // Belső él: 3/8 * (A + B) + 1/8 * (C + D)
      int f1Idx = adjFaces.get(0);
      int f2Idx = adjFaces.get(1);
      int c1 = getOppositeVertex(currentFaces.get(f1Idx), u, v);
      int c2 = getOppositeVertex(currentFaces.get(f2Idx), u, v);
      PVector pC = currentVertices.get(c1);
      PVector pD = currentVertices.get(c2);

      ep.add(PVector.mult(PVector.add(pA, pB), 3.0f / 8.0f));
      ep.add(PVector.mult(PVector.add(pC, pD), 1.0f / 8.0f));
    } else {
      // Határoló él (Boundary): 1/2 * (A + B)
      ep.add(PVector.mult(PVector.add(pA, pB), 0.5f));
    }
    edgePoints.put(key, ep);
  }

  // 3. Meglévő csúcsok új pozícióinak (Even vertices) számítása
  ArrayList<PVector> newVertexPositions = new ArrayList<PVector>();
  for (int i = 0; i < numVerts; i++) {
    PVector oldV = currentVertices.get(i);
    HashSet<Integer> neighbors = vertexNeighbors.get(i);
    int n = neighbors.size();

    if (n == 0) {
      newVertexPositions.add(oldV.copy());
      continue;
    }

    // Loop béta súlyképlet a csúcs valenciája (n) alapján
    float beta;
    if (n == 3) {
      beta = 3.0f / 16.0f;
    } else {
      beta = (5.0f / 8.0f - pow(3.0f / 8.0f + 0.25f * cos(TWO_PI / n), 2.0f)) /
n;
    }

    PVector sumNeighbors = new PVector(0, 0, 0);
    for (int neighborIdx : neighbors) {
```

```
      sumNeighbors.add(currentVertices.get(neighborIdx));
    }

    PVector newV = new PVector(0, 0, 0);
    newV.add(PVector.mult(oldV, 1.0f - n * beta));
    newV.add(PVector.mult(sumNeighbors, beta));

    newVertexPositions.add(newV);
  }

  // Csúcsok listájának összeállítása (régi módosítottak + új él-pontok)
  ArrayList<PVector> nextVertices = new ArrayList<PVector>();
  for (PVector v : newVertexPositions) {
    nextVertices.add(v);
  }

  HashMap<String, Integer> edgePointIndices = new HashMap<String, Integer>();
  for (String key : edgePoints.keySet()) {
    edgePointIndices.put(key, nextVertices.size());
    nextVertices.add(edgePoints.get(key));
  }

  // 4. Új háromszöges lapok generálása (1 háromszögből 4 darab lesz)
  ArrayList<ArrayList<Integer>> nextFaces = new
ArrayList<ArrayList<Integer>>();
  for (int fIdx = 0; fIdx < numFaces; fIdx++) {
    ArrayList<Integer> f = currentFaces.get(fIdx);
    int v0 = f.get(0);
    int v1 = f.get(1);
    int v2 = f.get(2);

    String e01Key = Math.min(v0, v1) + "_" + Math.max(v0, v1);
    String e12Key = Math.min(v1, v2) + "_" + Math.max(v1, v2);
    String e20Key = Math.min(v2, v0) + "_" + Math.max(v2, v0);

    int ep01 = edgePointIndices.get(e01Key);
    int ep12 = edgePointIndices.get(e12Key);
    int ep20 = edgePointIndices.get(e20Key);

    // 4 kis háromszög felépítése
    addTriangle(nextFaces, v0, ep01, ep20);
    addTriangle(nextFaces, ep01, v1, ep12);
    addTriangle(nextFaces, ep20, ep12, v2);
    addTriangle(nextFaces, ep01, ep12, ep20);
  }

  currentVertices = nextVertices;
  currentFaces = nextFaces;
}

// Segédfüggvény háromszög hozzáadásához
void addTriangle(ArrayList<ArrayList<Integer>> faces, int a, int b, int c) {
```

```
  ArrayList<Integer> t = new ArrayList<Integer>();
  t.add(a);
  t.add(b);
  t.add(c);
  faces.add(t);
}

// Segédfüggvény a háromszög harmadik (éggel szembeni) csúcsának megtalálásához
int getOppositeVertex(ArrayList<Integer> face, int u, int v) {
  for (int idx : face) {
    if (idx != u && idx != v) {
      return idx;
    }
  }
  return -1;
}

// --- PShape felépítése a GPU számára ---
void updateMeshShape() {
  meshShape = createShape();
  meshShape.beginShape(TRIANGLES);

  if (showWireframe) {
    meshShape.stroke(100, 200, 255);
  } else {
    meshShape.noStroke();
  }
  meshShape.fill(40, 90, 140, 220);

  for (ArrayList<Integer> f : currentFaces) {
    if (f.size() == 3) {
      addVert(f.get(0));
      addVert(f.get(1));
      addVert(f.get(2));
    }
  }
  meshShape.endShape();
}

void addVert(int idx) {
  PVector v = currentVertices.get(idx);
  meshShape.vertex(v.x, v.y, v.z);
}

// Kirajzolásért felelős függvény
void draw() {
  background(30);
  lights();

  pushMatrix();
  translate(width / 2, height / 2, 0);
  scale(zoom);
```

```
  rotateX(rotX);
  rotateY(rotY);

  if (meshShape != null) {
    shape(meshShape);
  }

  popMatrix();

  // --- 2D HUD ---
  hint(DISABLE_DEPTH_TEST);
  fill(255);
  textSize(16);
  text("Loop Subdivision szint: " + subLevel, 20, 30);
  text("1-es / 2-es gomb: Szint növelés / csökkentés", 20, 55);
  text("3-as gomb: Háló / Sima felület váltás", 20, 80);
  text("Fel/Le nyíl: Közelítés / Távolodás", 20, 105);
  text("Forgatás: kattints és húzd az egeret", 20, 130);
  hint(ENABLE_DEPTH_TEST);
}
```

Főbb változtatások:

1. **Loop szabályok**: A catmullClarkStep() helyét átvette a loopSubdivisionStep(), ami a Loop-féle maszkokat használja (élpontok súlyozása 3/8 és 1/8 arányban, csúcsok frissítése a 1-ring szomszédok és a csúcs valenciájától függő β paraméter alapján).
2. **Háromszögelés betöltéskor**: A Loop algoritmus alapvetően háromszögekre épül. A loadOBJ metódus mostantól automatikusan felbontja a négyszögű (quad) vagy sokszögű elemeket is háromszögekre, így nem lesz hiba a háló topológiájában.
3. **Rekurzív finomítás**: Minden egyes Loop lépésben minden egyes háromszög pontosan 4 darab kisebb háromszögre esik szét, gyönyörű, sima (C^2 / C^1 folytonos) limitfelületet eredményezve.

Sources

1. 13.5 Subdivision Surfaces
2. Loop subdivision surface - Wikipedia
